

DAY 03

WEEK LECTURE COURSE - DAY 03

서버 페칭 & Route Handler API 그리고 영화 검색 서비스

비동기 서버 데이터 수급과 Proxy API 아키텍처

Target: 대학 1학년 및 취업훈련생 입문 과정

Framework: Next.js 14/15 + Tailwind CSS

Date: 2026 Curriculum Edition

1. 목차 (Table of Contents)

- 학습 목표
- 도입: 서버에서의 데이터 패칭 vs 브라우저에서의 데이터 패칭
- 개념 개론: Next.js의 간결한 Fetch API와 서버 컴포넌트
- 핵심 기술: Route Handler (API 라우트)와 환경 변수 (`.env`)
- 실습: Mini-Project 3 - 영화 검색 익스플로러 (`/movies`)
- 실제 사례 연구: API Key 보안과 프록시 서버
- 결론 및 활용
- 강사 첨언 가이드 (Instructor Notes)

2. 학습 목표 (Learning Objectives)

1. React의 `useEffect` 데이터 페칭과 Next.js의 서버 컴포넌트 비동기(Async) 데이터 페칭의 차이 및 장점을 이해한다.
2. 서버 컴포넌트 내부에서 데이터를 요청할 때 보안 정보(API Key 등)가 클라이언트에 노출되지 않는 이유를 설명할 수 있다.
3. API Route Handler (`route.ts`)를 구축하여 프론트엔드가 자체 백엔드 API 역할을 하도록 코드를 설계한다.
4. `.env` 파일을 활용하여 중요한 환경 변수를 관리하는 법을 배운다.

3. 도입: 서버 페칭 vs 클라이언트 페칭

리액트만 사용할 때는 데이터를 가져올 때 항상 화면이 먼저 그려진 다음, 빈 로딩 스피너를 보여주며 브라우저 (클라이언트)에서 `fetch` 를 날렸습니다. 하지만 Next.js 서버 컴포넌트에서는 주방장(서버)이 요리를 내어주기 전에 식재료를 먼저 가져와서 조리된 결과물(데이터가 박힌 HTML)을 내보냅니다.

💡 직관적인 비유로 이해하는 데이터 요청 흐름

- 클라이언트 데이터 페칭 (React 방식)
 - 손님이 식당 테이블에 앉습니다. 테이블에 설치된 태블릿(브라우저)을 누르고 음식이 나오기를 기다립니다. 태블릿이 주방에 요청을 보내면, 한참 뒤에 반찬이 하나씩 옵니다. 화면이 깜빡이고 로딩이 길어집니다.

다.

- **서버 데이터 페칭 (Next.js 방식)**

- 손님이 자리에 앉자마자 밑반찬과 요리(데이터가 채워진 HTML)가 테이블에 이미 세팅되어 있습니다. 서버 내부의 백엔드 망에서 초고속으로 데이터를 미리 수급해 왔기 때문에 사용자 브라우저는 API 서버와 직접 통신하느라 대기할 필요가 없습니다.

4. 개념 개론: Next.js의 서버 페칭과 환경 변수

Next.js 서버 컴포넌트는 컴포넌트 함수 자체에 `async` 키워드를 붙이고 내부에서 바로 `await fetch()` 를 할 수 있습니다.

```
// 서버 컴포넌트 예시
export default async function MoviePage() {
  const res = await fetch('https://api.example.com/movies');
  const movies = await res.json();

  return (
    <ul>
      {movies.map(movie => <li key={movie.id}>{movie.title}</li>)}
    </ul>
  );
}
```

이 방식은 React의 `useState` 도, `useEffect` 도, `isLoading` 상태값도 필요 없는 혁신적인 단순함을 제공합니다.

환경 변수 (Environment Variables)

- `.env.local` : 로컬 개발 환경에서 사용할 비밀 키들을 저장하는 파일입니다.
- **보안 규칙:** Next.js에서는 `NEXT_PUBLIC_` 접두사를 붙이지 않은 변수는 오직 **서버 컴포넌트 또는 API Route Handler**에서만 접근 가능하며, 브라우저 자바스크립트 코드에는 절대 포함되지 않아 안전합니다.

5. 핵심 기술: Route Handler (API 라우팅)

만약 클라이언트 컴포넌트(브라우저)에서 외부 API를 호출해야 하는데 API 키 노출이 걱정되거나 CORS(이종 오리진 리소스 공유) 에러가 발생한다면 어떻게 해야 할까요? Next.js의 **Route Handler**를 사용하여

`/api/movies/route.ts` 같은 내부 서버 API 엔드포인트를 개설할 수 있습니다.

브라우저는 이 내부 API로 요청을 보내고, Next.js 서버가 대리인(Proxy)이 되어 외부 API에 안전하게 접근해 결과를 반환합니다.

6. 실습 - Mini-Project 3: 영화 검색 익스플로러

이 실습에서는 로컬 가상 영화 데이터베이스(`lib/movies.ts`)를 설계하고, 이를 노출하는 API Route Handler(`app/api/movies/route.ts`)와, 서버 컴포넌트를 활용해 데이터를 검색하고 필터링하는 영화 탐색기 페이지(`/movies`)를 제작합니다.

1) 가상 데이터 파일 생성: `lib/movies.ts`

`C:/Users/User/.gemini/antigravity/scratch/nextjs-lecture-portal/lib/movies.ts` 경로에 생성합니다.

```

export interface Movie {
  id: number;
  title: string;
  genre: string;
  rating: number;
  releaseYear: number;
  overview: string;
  image: string; // gradient placeholder
}

export const mockMovies: Movie[] = [
  {
    id: 1,
    title: "인셉션 (Inception)",
    genre: "SF / 스릴러",
    rating: 9.3,
    releaseYear: 2010,
    overview: "타인의 꿈에 침입해 무의식에 새로운 정보를 심는 작전을 수행하는 요원들의",
    image: "from-indigo-600 to-purple-800"
  },
  {
    id: 2,
    title: "인터스텔라 (Interstellar)",
    genre: "SF / 드라마",
    rating: 9.1,
    releaseYear: 2014,
    overview: "세계 식량 위기 속에서 인류의 새로운 터전을 찾아 wormhole을 통해 우주로 나아가",
    image: "from-blue-900 to-slate-950"
  },
  {
    id: 3,
    title: "다크 나이트 (The Dark Knight)",
    genre: "액션 / 느와르",
    rating: 9.5,
    releaseYear: 2008,
    overview: "고담시를 위협하는 혼돈의 지배자 조커에 맞서 도덕적 딜레마에 빠진 배트맨의",
    image: "from-slate-800 to-zinc-950"
  },
  {
    id: 4,
    title: "라라랜드 (La La Land)",
    genre: "로맨스 / 뮤지컬",
    rating: 8.9,
  }
]

```

```

    releaseYear: 2016,
    overview: "재즈 피아니스트와 배우 지망생의 꿈과 사랑을 그린 아름답고 애절한 음악 영
    image: "from-pink-600 to-rose-900"
  },
  {
    id: 5,
    title: "기생충 (Parasite)",
    genre: "스릴러 / 블랙코미디",
    rating: 9.2,
    releaseYear: 2019,
    overview: "전원 백수인 기택네 장남이 박사장네 과외 면접을 보러 가며 시작되는 두 가족
    image: "from-emerald-700 to-teal-950"
  },
  {
    id: 6,
    title: "어바웃 타임 (About Time)",
    genre: "로맨스 / SF",
    rating: 8.8,
    releaseYear: 2013,
    overview: "가문의 비밀인 시간 여행 능력을 깨닫게 된 주인공이 첫눈에 반한 여인과의 사
    image: "from-orange-500 to-red-700"
  }
];

```

2) Route Handler 생성: `app/api/movies/route.ts`

이 파일은 클라이언트가 쿼리를 날렸을 때 서버 사이드에서 필터링해서 JSON 데이터를 내려주는 REST API 엔드포인트 역할을 합니다.

```
import { NextResponse } from 'next/server';
import { mockMovies } from '@lib/movies';

export async function GET(request: Request) {
  // URL에서 검색어(query) 추출
  const { searchParams } = new URL(request.url);
  const query = searchParams.get('q')?.toLowerCase() || '';

  // 1초간의 네트워크 지연 시간 시뮬레이션 (RSC 로딩 효과 체험용)
  await new Promise((resolve) => setTimeout(resolve, 800));

  if (!query) {
    return NextResponse.json(mockMovies);
  }

  const filtered = mockMovies.filter(
    (movie) =>
      movie.title.toLowerCase().includes(query) ||
      movie.genre.toLowerCase().includes(query)
  );

  return NextResponse.json(filtered);
}
```

3) 영화 검색 페이지 생성: `app/movies/page.tsx`

이 페이지는 검색 인풋 컴포넌트(Client Side)와 영화 리스트 컴포넌트가 결합된 모던 검색기입니다.

```

'use client';

import { useState, useEffect } from 'react';
import Link from 'next/link';

interface Movie {
  id: number;
  title: string;
  genre: string;
  rating: number;
  releaseYear: number;
  overview: string;
  image: string;
}

export default function MovieExplorerPage() {
  const [movies, setMovies] = useState<Movie[]>([]);
  const [searchQuery, setSearchQuery] = useState('');
  const [loading, setLoading] = useState(true);

  // 영화 데이터 검색 페칭 함수
  const fetchMovies = async (query: string) => {
    setLoading(true);
    try {
      const res = await fetch(`/api/movies?q=${encodeURIComponent(query)}`);
      const data = await res.json();
      setMovies(data);
    } catch (e) {
      console.error('영화 데이터를 가져오는 데 실패했습니다.', e);
    } finally {
      setLoading(false);
    }
  };

  // 타이핑 시 0.4초 딜레이 후 자동 API 호출 (디바운싱 유사 구현)
  useEffect(() => {
    const timer = setTimeout(() => {
      fetchMovies(searchQuery);
    }, 300);

    return () => clearTimeout(timer);
  }, [searchQuery]);

```



```

return (
  <div className="min-h-screen bg-slate-950 text-slate-100 selection:bg-teal-500">
    <div className="max-w-5xl mx-auto space-y-8">

      {/* 상단 헤더 */}
      <div className="space-y-1">
        <Link href="/" className="text-xs text-teal-400 hover:underline font-sans">
          &larr; 홈으로 돌아가기
        </Link>
        <h1 className="text-3xl font-extrabold tracking-tight bg-gradient-to-r from-teal-400 to-teal-600">
          Movie Explorer
        </h1>
        <p className="text-slate-500 text-sm">
          Next.js API Route Handler 서버를 경유하여 실시간 필터링된 영화 데이터를 호스팅합니다.
        </p>
      </div>

      {/* 검색창 인풋 */}
      <div className="relative max-w-md">
        <input
          type="text"
          value={searchQuery}
          onChange={(e) => setSearchQuery(e.target.value)}
          placeholder="영화 제목 또는 장르를 검색하세요..."
          className="w-full bg-slate-900 border border-slate-800 rounded-2xl px-4 py-2"
        />
        <div className="absolute left-4 top-1/2 -translate-y-1/2 text-slate-500">
          <svg xmlns="http://www.w3.org/2000/svg" fill="none" viewBox="0 0 24 24">
            <path strokeLinecap="round" strokeLinejoin="round" d="M21 21l-5.197-5.197" />
          </svg>
        </div>
      </div>

      {/* 결과 디스플레이 영역 */}
      {loading ? (
        <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-6">
          {[1, 2, 3].map((n) => (
            <div key={n} className="bg-slate-900/50 border border-slate-900 rounded-2xl p-6">
              <div className="space-y-4">
                <div className="h-6 bg-slate-800 rounded-md w-2/3" />
                <div className="h-4 bg-slate-800 rounded-md w-1/3" />
                <div className="space-y-2 pt-4">
                  <div className="h-3 bg-slate-800 rounded w-full" />
                  <div className="h-3 bg-slate-800 rounded w-5/6" />
                </div>
              </div>
            </div>
          ))}
        </div>
      ) : (
        <div>
          <h2>영화 목록</h2>
          <table>
            <thead>
              <tr>
                <th>영화 제목</th>
                <th>장르</th>
                <th>평점</th>
                <th>연출자</th>
              </tr>
            </thead>
            <tbody>
              <tr>
                <td>The Shawshank Redemption</td>
                <td>드라마</td>
                <td>9.3</td>
                <td>Frank Darabont</td>
              </tr>
              <tr>
                <td>The Godfather</td>
                <td>범죄</td>
                <td>9.2</td>
                <td>Francis Ford Coppola</td>
              </tr>
              <tr>
                <td>The Dark Knight</td>
                <td>액션</td>
                <td>9.0</td>
                <td>Christopher Nolan</td>
              </tr>
              <tr>
                <td>The Godfather Part II</td>
                <td>범죄</td>
                <td>9.0</td>
                <td>Francis Ford Coppola</td>
              </tr>
              <tr>
                <td>The Silence of the Lambs</td>
                <td>범죄</td>
                <td>8.8</td>
                <td>Jonathan Demme</td>
              </tr>
            </tbody>
          </table>
        </div>
      )}
    </div>
  </div>
)

```

```

        </div>
      </div>
      <div className="h-8 bg-slate-800 rounded-md w-1/4" />
    </div>
  )))
</div>
) : movies.length === 0 ? (
  <div className="text-center py-24 bg-slate-900/20 border border-slate-900">
    <span className="text-4xl">🔍</span>
    <p className="text-slate-500 text-sm mt-4">검색 조건에 맞는 영화를 찾을 수 없습니다.</p>
  </div>
) : (
  <div className="grid sm:grid-cols-2 lg:grid-cols-3 gap-6">
    {movies.map((movie) => (
      <div
        key={movie.id}
        className="bg-slate-900 border border-slate-800 rounded-2xl overflow-hidden"
      >
        {/* 영화 포스터 대용 비주얼 그라데이션 박스 */}
        <div className={`h-40 bg-gradient-to-br ${movie.image} p-6 flex flex-direction-reverse`}>
          <div className="absolute inset-0 bg-black/10 group-hover:bg-black/20"></div>
          <span className="bg-slate-950/80 backdrop-blur text-[10px] font-medium">
            {movie.genre}
          </span>
          <div className="flex justify-between items-end z-10">
            <span className="text-xs text-slate-300 font-medium">{movie.releaseYear}</span>
            <span className="text-amber-400 text-xs font-bold bg-slate-950/80 backdrop-blur">
              ★ {movie.rating.toFixed(1)}
            </span>
          </div>
        </div>
      </div>
    ))}

    {/* 영화 소개 본문 */}
    <div className="p-6 flex-grow flex flex-col justify-between space-y-2">
      <div className="space-y-2">
        <h3 className="font-bold text-lg group-hover:text-teal-400 transition">영화 소개</h3>
        <p className="text-xs text-slate-400 leading-relaxed line-clamp-3">
          {movie.overview}
        </p>
      </div>
      <button
        onClick={() => alert(`[${movie.title}] 영화 상세 소개 페이지가 있습니다.`)}
        className="w-full py-2 bg-slate-950 border border-slate-800 group-hover:bg-slate-900"
      >
        영화 상세 소개
      </button>
    </div>
  </div>
)

```

```

        상세보기
      </button>
    </div>
  </div>
)}}
</div>
)}
</div>
</div>
);
}

```

7. 실제 사례 연구: API Key 보안과 프록시 서버

기상청 오픈 API나 네이버 뉴스 API 등을 연동할 때, 브라우저에서 직접 API Key를 담아 발송하면 해커가 개발자 도구의 네트워크 탭만 열어봐도 비밀 키를 탈취할 수 있습니다.

- **실무 해결책:** 기상청 API Key를 `.env` 파일에 서버 전용 변수로 숨겨놓습니다. 클라이언트는 우리 Next.js 서버 주소인 `/api/weather` 로만 요청하고, 외부 API와의 실제 요청은 Next.js 서버가 백그라운드에서 키를 담아 안전하게 수행합니다. 이것이 바로 **프록시(Proxy) 서버 아키텍처**입니다.

8. 결론 및 활용

- **요약:** Next.js 서버 컴포넌트는 간결하게 비동기 함수로 작성하여 데이터를 직접 가져올 수 있습니다. 브라우저와 통신이 빈번한 작업은 API Route Handler를 이용해 보안성을 유지하며 연결합니다.
- **숙제:** `lib/movies.ts` 에 본인이 좋아하는 영화 데이터를 1~2개 추가하고, 평점 9.0 이상만 보기 버튼 필터를 `/movies` 페이지에 구현해 봅니다.

9. 강사 첨언 가이드 (Instructor Notes)

💡 TIP

수업 진행 팁

- **00m ~ 30m:** React에서 `fetch` 할 때 겪었던 수많은 코드 라인수(State 3개 생성, `useEffect` 연동 등)와 Next.js 서버 페칭 방식의 라인 수를 칠판에 비교해 그리며 학생들을 전율케 하십시오.
- **30m ~ 50m:** 보안의 관점에서 `.env` 의 가치와 네트워크 탭에 키가 노출되는 원리를 브라우저 개발자 도구를 켜서 시연하십시오.
- **50m ~ 120m:** 영화 탐색기 프로젝트를 실습하고 API 서버의 딜레이 체감을 확인하며 UI 스켈레톤 로더 (Skeleton UI)의 필요성을 설명하세요.

⚠ WARNING

자주 발생하는 에러 및 대처법

1. **서버 컴포넌트와 상대 경로 `fetch` 에러:** 서버 컴포넌트 내부에서 `fetch('/api/movies')` 와 같이 상대 경로로 페칭하면 빌드 또는 구동 타임에 에러가 납니다. 서버 컴포넌트는 자신이 구동 중인 로컬호스트 주소를 모릅니다. 따라서 서버 페칭 시에는 외부 절대 주소를 쓰거나, 내부 파일일 경우 API 요청 없이 `lib/movies.ts` 에서 함수로 직접 호출하여 DB 읽듯이 연동하는 설계(Best Practice)가 정석임을 상기시켜 주세요. (본 예제는 로직 이해를 위해 클라이언트 측에서 Route Handler를 호출하도록 설계되어 안전합니다.)